

CKA Intensive

Day 3 — Scheduling, Storage, RBAC, Helm, Kustomize, HPA

Target: **Kubernetes v1.32**

The longest day. Helm + Kustomize + HPA are new in the 2024 curriculum — do not skip.

Today

1. **Morning quiz** (10 min, oral)
2. Scheduling (selectors, affinity, taints, topology spread)
3. Resource requests / limits / QoS
4. **Lab 6** — scheduling
5. Storage (Volumes, PV/PVC, StorageClass, dynamic provisioning, access modes)
6. **Lab 5** — storage
7. ResourceQuota + LimitRange
8. **Lab 5b** — quotas
9. RBAC + ServiceAccounts + kubeconfig
10. **Lab 6b** — RBAC
11. **Helm + Lab 6c**
12. **Kustomize + Lab 6d**
13. **HPA + Lab 6e**

Scheduling — what the scheduler actually does

For each unscheduled pod:

1. FILTER — drop ineligible nodes:
 - taints without matching toleration
 - failed nodeSelector / nodeAffinity required
 - insufficient resources (requests > node.allocatable – used)
 - port conflict (hostPort)
 - volume affinity (PV → zone)
2. SCORE — rank remaining nodes:
 - LeastRequestedPriority (favor empty nodes)
 - BalancedResourceAllocation (favor balanced cpu/mem use)
 - ImageLocalityPriority (favor nodes that already have the image)
 - NodeAffinity preferred (soft)
 - InterPodAffinity / Anti-affinity
 - TopologySpread
3. BIND — write pod.spec.nodeName

The scheduler is **stateless** in the long run — anything you teach it is in `pod.spec.*`.

nodeSelector — the simplest filter

```
spec:
  nodeSelector:
    disk: ssd
    region: us-east-1
```

Pod schedules ONLY on nodes where every label matches (AND of all keys).

```
k label node cka-worker disk=ssd region=us-east-1
k get nodes --show-labels
```

Built-in labels worth knowing: `kubernetes.io/hostname`, `kubernetes.io/os`, `kubernetes.io/arch`, `topology.kubernetes.io/zone`, `topology.kubernetes.io/region`, `node-role.kubernetes.io/control-plane`.

nodeAffinity — required vs preferred

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:      # HARD — filter
      nodeSelectorTerms:
        - matchExpressions:
            - { key: kubernetes.io/os, operator: In, values: [linux] }
            - { key: disk, operator: In, values: [ssd, nvme] }      # OR within values
        preferredDuringSchedulingIgnoredDuringExecution:      # SOFT — score boost
          - weight: 50
            preference:
              matchExpressions:
                - { key: topology.kubernetes.io/zone, operator: In, values: [us-east-1a] }
          - weight: 100
            preference:
              matchExpressions:
                - { key: gpu, operator: Exists }
```

Operators: **In**, **NotIn**, **Exists**, **DoesNotExist**, **Gt**, **Lt**.

IgnoredDuringExecution = it won't evict you if labels change after you're scheduled. There is no "required during execution" in stable.

podAffinity / podAntiAffinity

Place pods near (or away from) **other pods**, based on the labels of those other pods.

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels: { app: web }
        topologyKey: kubernetes.io/hostname           # "different nodes"
```

"No two `app=web` pods on the same hostname." Classic HA pattern: spread web across nodes.

`topologyKey` $\neq$ hostname:

- `kubernetes.io/hostname` → per-node spread
- `topology.kubernetes.io/zone` → per-zone spread

⚠ **Performance:** podAffinity is expensive at scale. Prefer **topology spread constraints** for spread.

Topology Spread Constraints

Modern, scalable spread.

```
spec:
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule # or ScheduleAnyway (soft)
      labelSelector:
        matchLabels: { app: web }
```

"Across zones, the count of `app=web` pods must not differ by more than 1."

Cheaper than podAntiAffinity, expresses intent better, scales to thousands of pods. Production default for HA spread.

Taints & tolerations

Taint on a node = "I don't accept pods unless they tolerate me."

```
k taint node cka-worker dedicated=db:NoSchedule
k taint node cka-worker dedicated:NoSchedule- # remove
```

Toleration on a pod = "I accept this taint."

```
spec:
  tolerations:
  - key: dedicated
    operator: Equal          # or Exists (matches any value)
    value: db
    effect: NoSchedule
    # tolerationSeconds: 60  # only for NoExecute – how long to stay after eviction triggers
```


Taint effects

Effect	New pods without toleration	Existing running pods without toleration
<code>NoSchedule</code>	rejected	stay running
<code>PreferNoSchedule</code>	soft repel	stay running
<code>NoExecute</code>	rejected	evicted (after <code>tolerationSeconds</code> if set)

Built-in taints worth knowing:

- `node-role.kubernetes.io/control-plane:NoSchedule` — why workload pods don't land on cp
- `node.kubernetes.io/not-ready:NoExecute` — when node goes NotReady, pods are evicted after 5 min
- `node.kubernetes.io/unreachable:NoExecute` — similar, for partition
- `node.kubernetes.io/disk-pressure:NoSchedule` — kubelet sets when disk fills

Resource requests & limits

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

what scheduler RESERVES
100 millicores = 0.1 vCPU
max enforced by kernel

- **CPU request** subtracted from node `allocatable` for scheduling
- **CPU limit** enforced via cgroup `cpu.cfs_quota` → **throttling** when exceeded (no kill)
- **Memory limit** → **OOM kill** when exceeded (container restarted)
- Memory has **no compressibility** — if you set it too low, you OOM. If you set it too high, you waste.

Units: `100m` = 0.1 CPU; `Mi` = mebibytes (1024²), `M` = megabytes (1000²) — use `Mi`.

QoS classes — set by the kubelet

Class	How you get it	Eviction order
Guaranteed	Every container has <code>requests == limits</code> for CPU AND memory	Last to be evicted
Burstable	At least one container has requests/limits, not Guaranteed	Middle
BestEffort	No requests or limits anywhere	First to be evicted under node pressure

Production: **set both requests and limits** on every container. Burstable for stateless web apps, Guaranteed for latency-sensitive services (databases, ingress controllers).

QoS shows up in `k describe pod` under `QoS Class`.

Eviction under node pressure

When a node hits `MemoryPressure` or `DiskPressure`, kubelet evicts pods in this order:

1. **BestEffort** pods first (any of them)
2. **Burstable** pods exceeding their request
3. **Burstable** pods within their request, sorted by priority
4. **Guaranteed** pods only as last resort

Eviction thresholds are kubelet flags (`--eviction-hard`), commonly `memory.available<100Mi,nodefs.available<10%`.

This is why you set requests **honestly**: under-promise gets you evicted; over-promise wastes capacity.

Lab 6 — Scheduling

→ `trainees/day3/labs/lab6-scheduling.md`

60 min.

- Label nodes; schedule via nodeSelector
- Taint a node; toleration to land a pod there
- nodeAffinity required + preferred
- podAntiAffinity to spread a Deployment
- Resource requests, watch a pod stay Pending due to "Insufficient cpu"

Storage — the layers



You almost always write only the **PVC**, and the **StorageClass** takes care of creating the PV.

Volume types you should recognize

Volume type	Lifetime	Use case
<code>emptyDir</code>	pod	scratch space, shared by containers in same pod
<code>hostPath</code>	node	mounts a host directory — careful, breaks portability
<code>configMap</code> / <code>secret</code>	pod	inject config files
<code>projected</code>	pod	combine multiple sources (SA token + CM + Secret) into one mount
<code>persistentVolumeClaim</code>	PV's reclaim policy	the production answer for state
<code>csi</code>	provisioner-dependent	modern external storage drivers

`emptyDir.medium: Memory` puts it in tmpfs (RAM-backed). Useful for ephemeral fast caches; counts against memory limit.

PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests: { storage: 5Gi }
  storageClassName: standard
```

`storageClassName: ""` = bind only to a PV with `storageClassName: ""` (static, no dynamic provisioning).
Omitting it entirely = use the **default** StorageClass.

The PVC stays `Pending` until a PV exists or is provisioned.

PV (static)

```
apiVersion: v1
kind: PersistentVolume
metadata: { name: data-static }
spec:
  capacity: { storage: 5Gi }
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""                # match a PVC with sc: ""
  hostPath:
    path: /mnt/data                  # use real CSI in production
```

Cluster admins (rarely) hand-create PVs. The 21st-century way is dynamic provisioning.

StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: rancher.io/local-path
parameters:
  type: ssd
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Provisioner names vary per backend: `ebs.csi.aws.com`, `pd.csi.storage.gke.io`, `disk.csi.azure.com`, `rancher.io/local-path` (kind).

Access modes (memorize)

Mode	Meaning	Common backends
<code>ReadWriteOnce</code> (RWO)	one node can mount RW	EBS, GCE PD, Azure Disk
<code>ReadOnlyMany</code> (ROX)	many nodes mount RO	rare
<code>ReadWriteMany</code> (RWX)	many nodes mount RW	NFS, CephFS, EFS, Azure Files
<code>ReadWriteOncePod</code> (RWOP)	exactly one Pod can mount RW	block storage with strict locking

Most cloud block storage is RWO. If you need RWX, you need a filesystem (NFS-like) backend.

Reclaim policy

What happens to a PV when its PVC is deleted?

Policy	Behavior
Delete	PV deleted + backing storage deleted (default for dynamic)
Retain	PV stays as Released ; you keep the data; cleanup is manual
Recycle	deprecated; do not use

Set on the **PV** (or via the StorageClass for dynamic).

Retain is what you want for anything you'd cry about losing.

volumeBindingMode

```
volumeBindingMode: WaitForFirstConsumer    # vs Immediate
```

- **Immediate:** PVC bound to a PV the moment it's created. PV may be in a zone the pod can't reach later — pod stuck.
- **WaitForFirstConsumer:** PV creation is delayed until a Pod that uses the PVC is scheduled. Scheduler picks a node FIRST, then storage is provisioned in *that* zone.

WaitForFirstConsumer is the right answer in multi-AZ clouds. Most managed StorageClasses default to it.

Volume expansion

```
# StorageClass  
allowVolumeExpansion: true
```

To grow a PVC, edit and bump `spec.resources.requests.storage`. Backend resizes. Online expansion supported by most CSI drivers.

Shrinking is not supported.

Lab 5 — Storage

→ `trainees/day3/labs/lab5-storage.md`

45 min.

- Default StorageClass + dynamic PVC + mount
- Static PV with `Retain`; verify PV remains after PVC delete
- subPath mount (single key from a ConfigMap into a file)
- Expand a PVC

ResourceQuota — namespace caps

Caps total resource use **across all pods + objects** in a namespace.

```
apiVersion: v1
kind: ResourceQuota
metadata: { name: ns-quota }
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 1Gi
    limits.cpu: "4"
    limits.memory: 2Gi
    pods: "5"
    persistentvolumeclaims: "2"
    requests.storage: 10Gi
    services.loadbalancers: "0"           # block LB creation
```

⚠ Once a quota tracks `requests.cpu`, **every pod must declare** requests/limits, or the API rejects the create. LimitRange fixes that.

LimitRange — automatic defaults

```
apiVersion: v1
kind: LimitRange
metadata: { name: defaults }
spec:
  limits:
    - type: Container
      default:      { cpu: 200m, memory: 256Mi } # → limits
      defaultRequest: { cpu: 100m, memory: 128Mi } # → requests
      max:          { cpu: "2", memory: 2Gi } # cap
      min:          { cpu: 50m, memory: 64Mi } # floor
```

When a pod is created without requests/limits, the LimitRange injects the defaults at admission time, so the quota is satisfied.

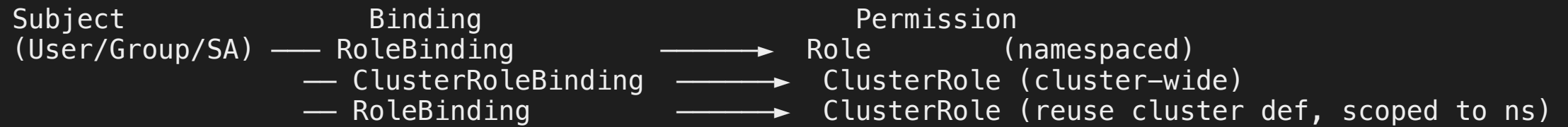
Quota + LimitRange together: "the namespace can't blow up, and users don't have to think about it."

Lab 5b — Quotas

→ `trainees/day3/labs/lab5b-quotas.md`

30 min. Apply a quota; watch an undefined-resources pod get rejected; fix it with a LimitRange.

RBAC — the model



- `Role` / `RoleBinding` = namespaced
- `ClusterRole` / `ClusterRoleBinding` = cluster-wide
- A `ClusterRole` referenced by a `RoleBinding` = use the cluster-defined rules, but only in that namespace

Subjects: `User`, `Group`, `ServiceAccount`. Users/Groups come from the authenticator (certs, OIDC); SAs are k8s objects.

Role + RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: team-a, name: pod-reader }
rules:
  - apiGroups: ["" ]                # core API group
    resources: [pods, pods/log]
    verbs: [get, list, watch]
  - apiGroups: [apps]
    resources: [deployments]
    verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { namespace: team-a, name: dev-binds }
subjects:
  - kind: ServiceAccount
    name: dev
    namespace: team-a
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Imperative RBAC

```
k create sa dev -n team-a
k create role pod-reader -n team-a \
  --verb=get,list,watch --resource=pods,pods/log,deployments.apps
k create rolebinding dev-binds -n team-a \
  --role=pod-reader --serviceaccount=team-a:dev

# verify (your most important debug tool):
k auth can-i list pods -n team-a \
  --as=system:serviceaccount:team-a:dev           # yes
k auth can-i delete pods -n team-a \
  --as=system:serviceaccount:team-a:dev           # no
```

`--as` impersonates a user/SA — works as long as your kubeconfig has `impersonate` permission (cluster-admin does).

ClusterRole / ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata: { name: pv-viewer }
rules:
  - apiGroups: ["" ]
    resources: [persistentvolumes, nodes]
    verbs: [get, list, watch]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: { name: ops-pv }
subjects: [{ kind: Group, name: ops, apiGroup: rbac.authorization.k8s.io }]
roleRef:
  kind: ClusterRole
  name: pv-viewer
  apiGroup: rbac.authorization.k8s.io
```

Cluster-scoped resources (Node, PV, ClusterRole, StorageClass, CRDs) **can only** be granted via ClusterRole.

Built-in ClusterRoles

Name	Permission
<code>cluster-admin</code>	full access (don't bind to humans)
<code>admin</code>	full RW within a namespace (incl. RoleBindings)
<code>edit</code>	RW most resources, no roles/bindings
<code>view</code>	RO

These are auto-created and updated by the controller-manager. Bind them via `RoleBinding` for namespace-scoped or `ClusterRoleBinding` for cluster-wide.

ServiceAccounts & tokens

Every namespace has a `default` SA. Pods auto-mount its token at `/var/run/secrets/kubernetes.io/serviceaccount/`.

Modern (1.22+): **projected, expiring SA tokens** (audience-scoped, time-bounded). The legacy long-lived Secret-backed tokens are deprecated; in 1.32 you generally generate them on demand.

```
k create token dev -n team-a --duration=1h      # short-lived token
k create token dev -n team-a --duration=24h --audience=external-system
```

Opt out of automounting:

```
spec:
  automountServiceAccountToken: false
```


kubeconfig anatomy

```
apiVersion: v1
kind: Config
clusters:
- name: prod
  cluster: { server: https://api.prod:6443, certificate-authority-data: <ca> }
users:
- name: alice
  user:
    client-certificate-data: <crt>
    client-key-data: <key>
contexts:
- name: prod-alice
  context: { cluster: prod, user: alice, namespace: team-a }
current-context: prod-alice
```

Three lists glued by name. The `current-context` picks one cluster + user + namespace tuple.

```
KUBECONFIG=/path/a:/path/b k config view --flatten > merged.kubeconfig
```

Lab 6b — RBAC

→ `trainees/day3/labs/lab6b-rbac.md`

45 min.

- Create SA + Role + RoleBinding for a namespace
- `kubectl auth can-i` verifications
- Cross-namespace with ClusterRole + RoleBinding
- Generate a kubeconfig for the SA token

Helm

The package manager for Kubernetes.

- A **chart** is a parameterized bundle of YAML templates
- A **release** is one install of a chart with a specific values set
- The Helm CLI talks to the apiserver; **no in-cluster Tiller** anymore (Helm 3+)
- Releases are stored as Secrets in the namespace they're installed in

2024 curriculum requires Helm. You will see it on the exam.

Helm chart anatomy

```
mychart/
├── Chart.yaml           ← metadata (name, version, appVersion, deps)
├── values.yaml          ← defaults
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── _helpers.tpl     ← named templates (partials)
│   └── NOTES.txt        ← shown after install
├── charts/              ← dependency charts (after `helm dep update`)
└── templates/tests/     ← test pods
```

Templates use Go template syntax + Sprig functions:

```
image: {{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}
```

Helm CLI — the verbs you need

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx

helm install web bitnami/nginx \
  --namespace web --create-namespace \
  --set service.type=ClusterIP \
  --set replicaCount=3 \
  --version 18.2.0

helm list -A
helm get values web -n web
helm get manifest web -n web | less          # rendered YAML

helm upgrade web bitnami/nginx -n web --set replicaCount=5
helm history web -n web
helm rollback web 1 -n web

helm uninstall web -n web
```

values.yaml override patterns

```
# inline
helm install web ./mychart --set image.tag=1.27 --set replicas=3

# from a file (production: keep this in git)
helm install web ./mychart -f values-prod.yaml

# multiple files; later wins
helm install web ./mychart -f base.yaml -f overrides.yaml

# render without installing – great for inspection/debugging
helm template web ./mychart -f values-prod.yaml | less
```

`helm template` is one of the most-underused features. Pipe to `kubectl apply -f -` if you ever want GitOps without Helm in-cluster.

Helm vs raw YAML — when

Use Helm when	Use raw YAML when
Consuming an off-the-shelf app (Prometheus stack, ingress-nginx)	Custom internal app you fully own
You need parameterized installs across envs	Two manifests are enough
Operator/community publishes charts only	Just one Deployment + Service
Lifecycle (upgrade/rollback) matters	One-shot bootstrap

The exam will likely ask you to: add a repo, install a chart, override values, upgrade, rollback. Practice this flow.

Lab 6c — Helm

→ `trainees/day3/labs/lab6c-helm.md`

30 min.

- Add bitnami repo, install nginx with custom values
- Inspect the release (`get values`, `get manifest`, `history`)
- Upgrade with `--set replicaCount=5`
- Roll back to revision 1

Kustomize

Built into `kubectl` — `kubectl apply -k <dir>`. No template engine, no Go templates. **Composition** instead of substitution.

```
overlays/prod/  
├── kustomization.yaml  
├── replicas.yaml      ← patch  
└── env.yaml           ← patch  
  
base/  
├── kustomization.yaml  
├── deployment.yaml  
└── service.yaml
```

The overlay references the base and adds patches on top. No magic syntax inside the YAML.

base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
commonLabels:
  app: web
```

overlays/prod/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: prod
resources:
- ../../base
patches:
- path: replicas.yaml
  target: { kind: Deployment, name: web }
images:
- name: nginx
  newTag: "1.28"
configMapGenerator:
```

Kustomize patch — strategic merge

```
# overlays/prod/replicas.yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 5
```

That's the whole patch. Kustomize merges this into the base Deployment, replacing only `spec.replicas`.

For surgical edits, use JSON6902:

```
patches:
- target: { kind: Deployment, name: web }
  patch: |
    - op: replace
      path: /spec/template/spec/containers/0/image
      value: nginx:1.28
```

Kustomize vs Helm

	Helm	Kustomize
Template engine	Yes (Go templates)	No — pure overlay
Release tracking	Built-in (history, rollback)	None (you track via git)
Conditionals / loops	Yes (powerful)	No (deliberate)
Learning curve	Steeper	Gentler
Lock-in	Charts are Helm-specific	Manifests stay as plain YAML
Common use	Distributed software (community charts)	Internal apps, environment overlays

Both, often: Helm renders the chart, Kustomize overlays your env-specific changes (`helm template | kustomize build`).

Lab 6d — Kustomize

→ `trainees/day3/labs/lab6d-kustomize.md`

30 min.

- One `base/` (deployment + service)
- Two overlays (`dev/`, `prod/`) — different replicas, different env, different namespace
- `kubectl apply -k overlays/dev/` then `prod/`

HPA — Horizontal Pod Autoscaler

Scales a Deployment / StatefulSet up/down based on metrics.

Prerequisites:

- `metrics-server` installed (`kubectl top pods` works)
- The target workload has `resources.requests` (HPA computes utilization as a percentage of request)

```
k apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
# on kind: edit deployment, add '--kubelet-insecure-tls'
k top nodes
k top pods -A
```

HPA v2 YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60          # scale to keep avg at 60% of request
    - type: Resource
      resource:
        name: memory
        target:
          type: AverageValue
          averageValue: 200Mi
  behavior:                               # scale rate control (v2)
    scaleDown:
      stabilizationWindowSeconds: 300
      policies: [{ type: Percent, value: 50, periodSeconds: 60 }]
    scaleUp:
      policies: [{ type: Pods, value: 4, periodSeconds: 30 }]
```

HPA — why it doesn't scale

Symptom in <code>describe hpa</code>	Cause
<code><unknown>/60%</code> for current CPU	metrics-server not installed, not Ready, or can't scrape kubelets
<code>0/60%</code> and pods are clearly busy	Pods missing <code>resources.requests.cpu</code> — no denominator
Bouncing replicas	<code>stabilizationWindowSeconds</code> too short; tune in <code>behavior</code>
Won't scale past N	Hit <code>maxReplicas</code> , or quota
Won't scale below N	Hit <code>minReplicas</code> , or PDB

`k describe hpa <name>` shows the events: "FailedGetResourceMetric" etc.

HPA + load gen

```
# Deployment with cpu request, 2 replicas
k create deploy web --image=nginx:1.27 --replicas=2 \
  --port=80 --dry-run=client -o yaml > web.yaml
# add resources.requests.cpu: 100m, apply
k apply -f web.yaml

k expose deploy web --port=80
k autoscale deploy web --min=2 --max=10 --cpu-percent=50

# load gen from a busy pod
k run hey --image=ghcr.io/rakyll/hey -- -z 5m -c 50 http://web

k get hpa web -w          # watch it scale
k top pods -l app=web
```

Lab 6e — HPA

→ `trainees/day3/labs/lab6e-hpa.md`

30 min.

- Install metrics-server (kind tweak)
- Deploy with cpu requests
- Create HPA via `k autoscale`
- Generate load; watch HPA scale up; let it cool; watch scale down

Day 3 wrap-up

You can now:

- Place pods deterministically with selectors, affinity, taints, topology spread
- Set requests/limits correctly and predict QoS / eviction
- Provision storage dynamically and statically
- Write tight RBAC and verify with `auth can-i`
- Install + upgrade + rollback with Helm
- Compose YAML with Kustomize bases + overlays
- Wire HPA against metrics-server with sane `requests`

Tonight:

- Skim `helm.sh/docs/topics/charts/`
- 1 killercoda **scheduling** scenario
- Re-do the RBAC lab from scratch on a fresh namespace